



From Address Blocks to Authorized Prefixes: Redesigning RPKI ROV with a Hierarchical Hashing Scheme for Fast and Memory-Efficient Validation

Zedong Ni, Computer Network Information Center, Chinese Academy of Sciences; and School of Cyber Science & Engineering, Southeast University; Yinbo Xu, Hui Zou, and Yanbiao Li, Computer Network Information Center, Chinese Academy of Sciences; and University of Chinese Academy of Sciences; Guang Cheng, School of Cyber Science & Engineering, Southeast University; and Purple Mountain Laboratories; Gaogang Xie, Computer Network Information Center, Chinese Academy of Sciences; and University of Chinese Academy of Sciences

<https://www.usenix.org/conference/nsdi25/presentation/ni>

**This paper is included in the
Proceedings of the 22nd USENIX Symposium on
Networked Systems Design and Implementation.**

April 28–30, 2025 • Philadelphia, PA, USA

978-1-939133-46-5

**Open access to the Proceedings of the
22nd USENIX Symposium on Networked
Systems Design and Implementation
is sponsored by**



From Address Blocks to Authorized Prefixes: Redesigning RPKI ROV with a Hierarchical Hashing Scheme for Fast and Memory-Efficient Validation

Zedong Ni^{1,2,#}, Yinbo Xu^{1,3,#}, Hui Zou^{1,3}, Yanbiao Li^{1,3,*}, Guang Cheng^{2,4}, and Gaogang Xie^{1,3,*}

¹Computer Network Information Center, Chinese Academy of Sciences

²School of Cyber Science & Engineering, Southeast University

³University of Chinese Academy of Sciences

⁴Purple Mountain Laboratories

Abstract

Route Origin Validation (ROV) with Route Origin Authorizations (ROAs), built on top of the Resource Public Key Infrastructure (RPKI), serves as the only formally standardized and production-grade defense mechanism against route hijackings in global interdomain routing infrastructures. However, the widespread adoption of RPKI has introduced escalating scalability challenges in validating high volumes of route messages against massive ROA entries.

In this paper, we attribute the performance bottleneck of existing ROV schemes to their underlying validation model, where the route is matched against rules in the form of address blocks. To overcome this bottleneck, we propose the *Authorized Prefix (AP) model* that enables validation at the prefix granularity, and redesign RPKI ROV based on this new model with a hierarchical hashing scheme named h^2 ROV. Extensive evaluations verify h^2 ROV's superiority over state-of-the-art approaches in IPv4, with a speedup of $1.7\times \sim 9.8\times$ in validation and a reduction of 49.3% $\sim$ 86.6% in memory consumption. System emulations using real-world network topologies further demonstrate h^2 ROV confines its impact to routing convergence to below 8.5% during update burst events, while reducing ROV-induced delays by 30.4% $\sim$ 64.7% compared to existing solutions.

1 Introduction

1.1 BGP Security and Route Origin Validation

The Border Gateway Protocol (BGP) [1] is a fundamental building block of the Internet, which enables reachability information exchange among autonomous systems (ASes). Despite its critical role in maintaining global connectivity, BGP exhibits inherent vulnerabilities to malicious exploitation, particularly route hijacking attacks that can propagate

invalid routing announcements across AS boundaries, causing persistent and large-scale disruptions to the global Internet.

To mitigate these threats, Route Origin Validation (ROV) has emerged as the sole standardized and production-ready security mechanism. Built upon the Resource Public Key Infrastructure (RPKI) [2], ROV utilizes cryptographically signed Route Origin Authorizations (ROAs) to establish a trust anchor for BGP route origination. Network operators implement ROV through router-side verification processes that cross-check advertised BGP routes against RPKI-validated ROAs, thereby identifying and filtering unauthorized origin AS claims.

1.2 Concerns about the Efficiency of ROV

While ROV significantly enhances the security of BGP route origination, its effectiveness remains constrained by incomplete global RPKI adoption. Notably, despite ROAs covering over 50% of BGP route prefixes, only approximately 12% of ASes actively enforce ROV in production networks [3]. This adoption gap persists even among operators with deployed ROAs, highlighting systemic barriers beyond mere infrastructure readiness. A longitudinal study [4] identifies operational efficiency concerns as the primary deterrent to ROV adoption.

ROV introduces two critical operational challenges for routers. First, the validation process imposes per-route computational overhead, requiring real-time verification of ROAs against BGP routes. Second, the sequential validation of bulk routing updates compounds latency, directly impacting BGP convergence—a key performance metric for global routing stability. Our experimental analysis reveals that state-of-the-art ROV implementations increase BGP convergence time by 8.0% $\sim$ 25.6% under update bursts¹, a frequent phenomenon driven by recurring route anomalies (e.g., misconfigurations and hijacks). Worse still, inefficient ROV introduces a new attack vector, enabling adversaries to overwhelm ROV-enforcing routers by amplifying validation overhead [6].

The first two authors contributed equally to this work. Zedong Ni participated in this study during his master's degree program at the Computer Network Information Center of the Chinese Academy of Sciences.

* Corresponding authors: lybmth@cnic.cn and xie@cnic.cn.

¹BGPMon [5] shows that route anomalies occur frequently in recent years, which may lead to update bursts.

Another concern arises from ROV’s escalating storage demands as RPKI adoption increases. Our empirical analysis demonstrates that contemporary ROV implementations exhibit memory footprints approaching parity with the core Routing Information Base (Table 3, Appendix D.1). This substantial overhead originates from two fundamental requirements: 1) maintaining synchronized replicas of cryptographically validated ROA entries, and 2) supporting specialized lookup structures for accelerated route validation processes. These combined components directly compete with allocation priorities for essential routing functions, creating an operational paradox where network operators face pressure to disable ROV - despite its security benefits - to preserve baseline routing stability.

1.3 Prior Arts and Limitations

Existing research has explored two dominant approaches to optimize ROV performance: hash-based and trie-based validation architectures. Hash-based methods [7] organize ROAs in hash tables, achieving memory efficiency by compressing prefix-to-AS mappings. Nevertheless, this approach increases validation latency, as resolving super/prefix containment requires multiple hash probes. Conversely, trie-based implementations [6–8] improve validation speed by structuring address blocks into hierarchical prefix trees, achieving $O(\log n)$ lookup times. However, they introduce higher memory overhead due to trie node proliferation.

Although existing optimizations partially mitigate ROV inefficiencies, our large-scale emulations demonstrate that both trie- and hash-based architectures exhibit critical scalability limitations under projected RPKI adoption trajectories. Specifically, the accelerating coverage of BGP routes by ROAs will substantially increase the proportion of ROV-validated routes. Counterintuitively, validation latency in state-of-the-art systems sharply increases with valid route volume. This divergence leads to an operational dilemma: improved security via RPKI adoption directly degrades real-time route processing capabilities.

1.4 Key Insights and Our Contributions

Regarding the aforementioned challenge in ROV with existing solutions, we conduct a comprehensive investigation to identify the root cause of their performance bottlenecks. Our systematic analysis reveals that the fundamental limitation stems from inherent deficiencies in the conventional validation model [9]. To address this critical issue, we propose a groundbreaking *Authorized Prefix (AP)* validation framework in Section 3, a novel paradigm that establishes two pivotal advancements: (1) superior operational efficiency compared to the traditional model especially when valid route volume is large, and (2) guaranteed equivalence in validation accuracy through formal verification.

Then, in Section 4, we present h^2ROV , a hierarchical hashing scheme for fast and memory-efficient ROV based on the *AP model*. It employs two hierarchical hash tables to manage authorized prefixes extracted from ROAs with bitmap encoding, enabling ROV at prefix granularity. We further propose two optimizations to improve its validation performance in IPv4 and IPv6 respectively. In addition, we validate h^2ROV ’s practical applicability through a reference implementation integrated with two widely-deployed open-source software routers, *FRRouting*² and *BIRD*³, in Section 5.

At last, in Section 6, we evaluate h^2ROV using real-world RPKI repositories, BGP routing tables and update traces from operational networks. Compared to state-of-the-art ROV solutions, h^2ROV reduces memory usage by 49.3% ~ 86.6% and accelerates IPv4 route validation by 1.7 ~ 9.8 times. System emulations using real-world network topologies further demonstrate h^2ROV mitigates ROV-induced routing convergence delays by up to 64.7%, outperforming conventional validation architectures under routing instability scenarios.

2 Background and Motivation

2.1 Route Origin Authorizations

As defined in RFC 9582 [10], an ROA is a cryptographically signed object, containing an ASN and a sequence of *ipAddrBlocks*. An *ipAddrBlock* encodes a tuple (IP Prefix, *maxLength*), encompassing all sub-prefixes of IP Prefix with prefix lengths up to the specified *maxLength*. We call such a tuple as an *address block (AB)*. The Replying Party (RP) retrieves ROAs from the RPKI repository and validates them locally. Then, all validated ROAs are distributed to routers via the RPKI-to-Router (RTR) protocol [11]. Routers receive these ROAs and extract the key authorization information to guide the ROV process. We define the extracted authorization information as ROV rules. Additionally, to clarify terminology, in this paper, references to ROAs specifically denote an AB paired with its authorized AS, represented by the tuple (IP Prefix, *maxLength*, ASN), abbreviated as (*pfx*, *ml*, *ao*).

2.2 ROV Model with Address Blocks

Current ROV schemes validate BGP routes using a list of ROV rules, where each ROV rule, extracted from received ROAs, can be represented as a tuple (*pfx*, *ml*, *ao*) comprising an AB and its authorized AS [9]. We define the *AB model* as the ROV model that uses AB-based ROV rules. Validity states of the *AB model* is defined as follows.

Definition 1 Let r denote a route, represented as a tuple (rp , ro), where rp is the route prefix and ro is the origin ASN.

²<https://frrouting.org/>

³<https://bird.network.cz/>

Prefix-to-ASN Mappings	
192.0.2.0/25, AS X	192.0.2.64/31, AS Z
192.0.2.64/26, AS Y	192.0.2.64/32, AS Z
192.0.2.64/27, AS Y	192.0.2.65/32, AS Z
192.0.2.96/27, AS Y	

Figure 1: Prefix-to-ASN Mappings.

List of ROV rules
192.0.2.0/25, 25, AS X
192.0.2.64/26, 27, AS Y
192.0.2.64/31, 32, AS Z

Figure 2: List of ROV rules.

Definition 2 Let $|p_i|$ denote the prefix length of p_i . $p_i \subseteq p_j$ means p_i is a sub-prefix of p_j .

Definition 3 Validity states: Given a list of ROV rules RL and a route r , the validity state can be *valid*, *invalid*, or *notFound*. *Valid* means $\exists R \in RL, R \text{ matches } r$. *Invalid* means $(\exists R \in RL, R \text{ covers } r) \wedge (\forall R \in RL, R \text{ covers } r \implies R \text{ does not match } r)$. *NotFound* means $\forall R \in RL, R \text{ does not cover } r$.

Definition 4 Cover/Match: Given an ROV rule R and a route r , $R \text{ covers } r$ means $r.rp \subseteq R.pfx$. $R \text{ matches } r$ means $R \text{ covers } r \wedge |r.rp| \leq R.ml \wedge r.ro = R.ao$.

In the *AB model*, two conditions are defined: *ExistCover* (*EC*), which finds an ROV rule that covers route r , and *CheckMatch* (*CM*), which examines whether the covering ROV rule R satisfies $R.ml \geq |r.rp| \wedge R.ao = r.ro$. If the *EC* condition fails, the ROV result is *notFound*. Otherwise, if both *EC* and *CM* conditions are met, the ROV result is *valid*. In this case, the *EC* and *CM* conditions are performed α times, where α ranges from 1 to n , with n being the number of ROV rules covering r in RL . If the *EC* condition is satisfied but the *CM* condition is not, the ROV result is *invalid*, and both conditions are performed n times. We abstract a decision tree from the *AB model*, namely the *CoverFirst* (*CF*) tree, as shown in Figure 3.

2.3 Hanging ROA and Bitmap Encoding

Hanging ROA (h-ROA) is a bitmap-based encoding scheme for ROAs [12]. It encodes authorized prefixes within ROAs into split sub-trees. In this way, an ROA is transformed into an ASN and a list of encoded sub-trees, each represented as a tuple (id, bm) . The id uniquely identifies each sub-tree, derived from the root prefix of that sub-tree. The operation of computing id can be defined as $ID(p, n)$, which concatenates a set bit ('1') with the first n bits of prefix p and converts the resulting binary string to a decimal value. The bm records authorized prefixes within the sub-tree. Each bit represents a prefix, where '1' indicates authorization and '0' otherwise. Due to its design of encoding authorized prefixes into bitmaps, h-ROA supports ROA updates at prefix level.

2.4 Motivation and A Running Example

However, the ROV with *AB model* is inherently limited. We use an example to illustrate the limitations of the *AB model*. There are seven prefixes assigned to three ASes, mapped in Figure 1, generating three ROV rules shown in Figure 2. For

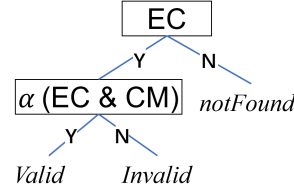


Figure 3: Decision tree of the *AB model*.

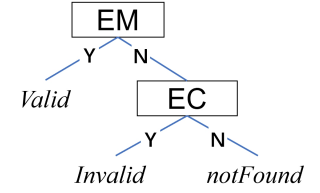


Figure 4: Decision tree of the *AP model*.

a route $(192.0.2.64/32, AS Z)$, the ROV-enforcing router determines its validity state as *valid*. In this example, the router performs three queries, sequentially checking whether each ROV rule covers the route prefix and ultimately identifying a matching ROV rule.

The *AB model*'s performance bottleneck lies in its coarse-grained address block management. The bitmap-based encoding enables fine-grained management of authorized prefixes and provides valuable insights for improving ROV. We propose the *Authorized Prefix* (*AP*) model, which uses authorized prefixes for route validation and builds on bitmap-based encoding to achieve fine-grained management of ROV rules. Through careful design and optimization, the *AP model* can overcome the performance bottlenecks of the *AB model*.

3 ROV Model with Authorized Prefixes

3.1 Model Definition

Instead of encoding an ROV rule as an *AB* and its authorized AS, the *AP model* maintains ROV rules at prefix granularity. Each ROV rule is encoded as a tuple (pfx, ao) comprising a prefix and its authorized AS. Validity states of the *AP model* remain the same as those of the *AB model*, while the definition of Cover/Match differs, as defined below.

Definition 5 Cover/Match: Given an ROV rule R and a route r , $R \text{ covers } r$ means $r.rp \subseteq R.pfx$. $R \text{ matches } r$ means $r.rp = R.pfx \wedge r.ro = R.ao$.

In the *AP model*, we define two conditions: *ExistCover* (*EC*), which finds an ROV rule that covers route r , and *ExistMatch* (*EM*), which finds an ROV rule that matches r . If the *EM* condition is satisfied, the ROV result is *valid*. Otherwise, the ROV result is *invalid* or *notFound*, depending on whether the *EC* condition is met. Both conditions are executed at most once. We abstract a decision tree from the *AP model*, namely the *MatchFirst* (*MF*) tree, as shown in Figure 4.

3.2 Cost Analysis of ROV

Cost of the *AB model*. To analyze the validation cost of the *AB model*, we define the execution costs of *EC* and *CM* as X_1 and Y_1 , as the complexities of these two conditions obviously differ. Then, the validation costs determining ROV

results are $\alpha X_1 + \alpha Y_1$, $nX_1 + nY_1$, and X_1 for *valid*, *invalid*, and *notFound* routes, respectively. Finally, we formalize the minimal cost of the *AB model* according to the *CF tree* based on the distribution across three validity states (with $\alpha = 1$ and $n = 1$ for minimal cost):

$$\begin{aligned} \text{cost}(CF) &= P_1(X_1 + Y_1) + P_2(X_1 + Y_1) + P_3X_1 \\ &= X_1 + (1 - P_3)Y_1 \end{aligned} \quad (1)$$

where P_1 , P_2 , and P_3 are the proportions of *valid*, *invalid*, and *notFound* routes, respectively, and their total equals 100%. Clearly, as RPKI deployment increases, the proportion of *notFound* routes decreases correspondingly. Therefore, the validation cost of the *AB model* increases.

Cost of the AP model. The costs of executing the *EC* and *EM* conditions are defined as X_2 and Y_2 . Thus, the validation costs determining ROV results are Y_2 , $X_2 + Y_2$, and $X_2 + Y_2$ for *valid*, *invalid*, and *notFound* routes, respectively. Finally, we formalize the cost of the *AP model* using the *MF tree* based on the distribution across three validity states:

$$\begin{aligned} \text{cost}(MF) &= P_1Y_2 + P_2(X_2 + Y_2) + P_3(X_2 + Y_2) \\ &= (1 - P_1)X_2 + Y_2 \end{aligned} \quad (2)$$

The validation cost of the *AP model* is affected by P_1 . As the proportion of *valid* routes increases, the validation cost decreases.

Cost comparison. We compare the validation costs of the two ROV models based on the previous analysis. In both models, the *EC* condition tries to find one or more ROV rules that cover a given route. This can be regarded as the *subset rank problem*, which has a time complexity of $O(\log n)$, where n is the number of ROV rules [13]. Therefore, we assign the same time consumption to X_1 and X_2 , denoted as $X_1 \equiv X_2 \equiv x$. The *CM* and *EM* conditions, on the other hand, can be performed in constant time, so we define $Y_1 \equiv Y_2 \equiv y$. Furthermore, it is clear that $x \gg y$. With these assumptions, we can formalize the difference between the minimum cost of $\text{cost}(CF)$ (where $n = 1$ and $\alpha = 1$) and the maximum cost $\text{cost}(MF)$ as follows:

$$\begin{aligned} \Delta(\text{cost}) &= \text{cost}_{\min}(CF) - \text{cost}_{\max}(MF) \\ &= X_1 + (1 - P_3)Y_1 - ((1 - P_1)X_2 + Y_2) \\ &= P_1x - P_3y \end{aligned} \quad (3)$$

Currently, $P_1 > P_3$, so the difference in cost between the two models is greater than 0, indicating that the maximum cost of the *AP model* is lower than the minimum cost of the *AB model*. Moreover, as RPKI gets more widely deployed, the proportion of *valid* routes (P_1) will increase and that of *notFound* routes (P_3) will decrease. This will further widen the cost difference between the two models.

3.3 Equivalence in Validation Accuracy

This subsection presents the proof that the *AP model* is equivalent to the *AB model* in terms of ROV outcomes.

Definition 6 Given an ROV rule $R(pfx, ml, ao)$, the set of prefixes contained in R is defined as: $AP_{\text{set}}(R) = \{p \mid p \subseteq R.pfx \wedge |p| \leq R.ml\}$.

Definition 7 Given a list of ROV rules RL_{AB} under the *AB model*, the *AP model* generates its list of ROV rules RL_{AP} as follows: $RL_{AP} = \{(pfx, ao) \mid pfx \in AP_{\text{set}}(R) \wedge ao = R.ao, \forall R \in RL_{AB}\}$

Theorem 1 Given an ROV rule $R(pfx, ml, ao)$ and a route prefix rp , $rp \not\subseteq R.pfx \iff \forall p \in AP_{\text{set}}(R), rp \not\subseteq p$.

The proof of this theorem is provided in Appendix B.

Theorem 2 For a given route $r(rp, ro)$, the ROV result under the *AB model* is identical to that under the *AP model*.

By Definition 3, if the ROV result for route $r(rp, ro)$ is *notFound* under the *AB model*, then $\forall R \in RL_{AB}, rp \not\subseteq R.pfx$. According to Theorem 1, we can infer that $\forall p \in AP_{\text{set}}(R), rp \not\subseteq p$ for all $R \in RL_{AB}$. Combining with Definition 7, $\forall R \in RL_{AP}, rp \not\subseteq R.pfx$, the ROV result for route r under the *AP model* is also *notFound*, and conversely.

If the ROV result for r is *valid* under the *AB model*, then $\exists R \in RL_{AB}, rp \subseteq R.pfx \wedge |rp| \leq R.ml \wedge ro = R.ao$. Let this ROV rule be MR . According to Definition 6, $rp \in AP_{\text{set}}(MR) \wedge ro = MR.ao$. Since $MR \in RL_{AB}$, by Definition 7, we deduce that $(rp, ro) \in RL_{AP}$. Consequently, $\exists R \in RL_{AP}, R.pfx = rp \wedge R.ao = ro$. Hence, the ROV result under the *AP model* is also *valid*, and conversely.

If the ROV result for r is *invalid* under the *AB model*, it is neither *valid* nor *notFound*. Since the *AB* and *AP models* agree on classifying routes as *valid* or *notFound*, the *AP model* also deem it neither *valid* nor *notFound*. Hence, the result under the *AP model* is *invalid*, and conversely.

In conclusion, given identical ROAs and routes, the validation results produced by the *AB model* and the *AP model* are equivalent.

4 h²ROV: Hierarchical Hashing-Based ROV

In this section, we introduce the basic version of our proposed ROV scheme, h²ROV. Subsequently, we introduce two optimizations specifically tailored for IPv4 and IPv6.

4.1 Maintaining ROV rules with Bitmaps

Instead of refining ROV under the limitations of the *AB model*, we propose a novel ROV scheme, h²ROV, which performs ROV based on the *AP model*. To achieve this, h²ROV employs the basic version of h-ROA for managing ROV rules at the prefix level, which sets the hanging levels as multiples of 5. Figure 5 illustrates how h²ROV maintains the ROV rules at the prefix level and performs ROV based on these finer-granularity

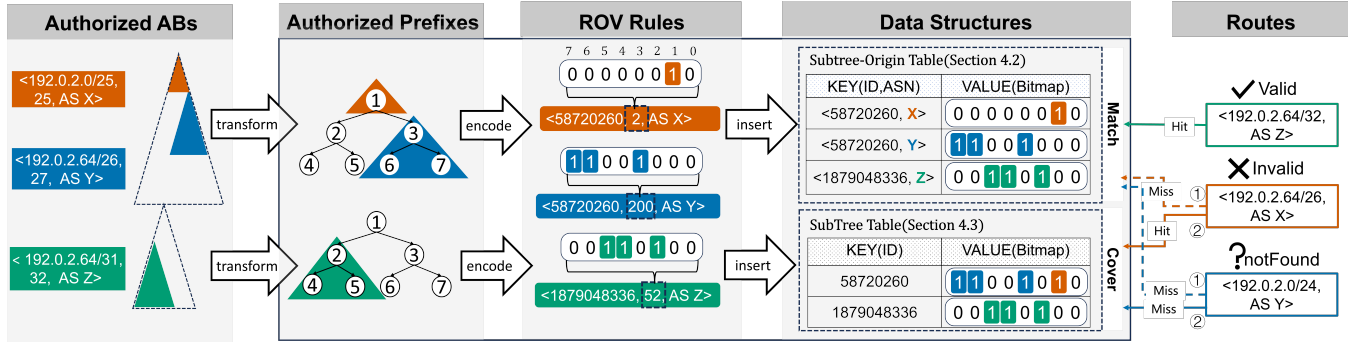


Figure 5: In h^2 ROV scheme, when the router receives ROAs, it first encodes them into a series of ROV rules and then inserts them into the corresponding data structures.

authorizations. For example, the ROA (192.0.2.64/26, 27, AS Y) (blue square) authorizes three prefixes for AS Y. h^2 ROV first locates the sub-tree containing these prefixes, rooted at 192.0.2.0/25. The sub-tree's id is calculated as 58720260, derived from the binary string formed by concatenating a set bit ('1') with the first 25 bits of 192.0.2.0. Then, h^2 ROV encodes the three authorized prefixes into the bitmap 11001000 (only the last 8 bits are shown, as the higher bits are 0). Bits 3, 6, and 7 are set to 1, corresponding to these prefixes. Similarly, ROAs (192.0.2.0/25, 25, AS X) (orange square) and (192.0.2.64/31, 32, AS Z) (green square) are encoded into (58720260, 00000010, AS X) and (1879048336, 00110100, AS Z). In summary, when routers receive ROAs, h^2 ROV transforms them into ROV rules, which are represented as (id, bm, ASN) tuples.

4.2 Matching the Route

Given a route (rp, ro), h^2 ROV first calculates the sub-tree's id as ID (rp, |rp|). If a bitmap is found in the SOT for (id, ro), h^2 ROV calculates the index of rp in the sub-tree as i and checks whether the i-th bit in the bitmap is set to 1. If the bit is set, it indicates that rp is authorized by an ROA for ro, and the route is *valid*. By leveraging the SOT, this matching process achieves a time complexity of $O(1)$.

As shown in Figure 5, the router encodes the fetched ROAs into three ROV rules: (58720260, 00000010, AS X), (58720260, 11001000, AS Y), and (1879048336, 00110100, AS Z). These entries are then inserted into the SOT as ((58720260, AS X), 00000010), ((58720260, AS Y), 11001000), and ((1879048336, AS Z), 00110100). For a received route (192.0.2.64/32, AS Z), h^2 ROV first calculates the sub-tree's id as 1879048336. It then identifies the index of the route within the sub-tree as 4. Next, h^2 ROV queries the SOT using the key (1879048336, AS Z) and retrieves the corresponding bitmap 00110100. Finally, h^2 ROV determines the route's validity state by checking whether the 4th bit in this bitmap is set to 1. In this case, the bit is set, confirming that the route is *valid*.

4.3 Covering the Route Prefix

For *valid* routes, the SOT enables direct matching. However, for *invalid* or *notFound* routes, where SOT alone cannot confirm status, further validation is needed. We deploy a secondary hash table STT to locate ROV rules covering the route prefix. The STT utilizes sub-tree's id as key and stores a bitmap union of all authorized prefixes within corresponding sub-tree.

When a route (rp, ro) fails SOT matching, h^2 ROV retrieves the bm in the STT using id=ID(rp, |rp|). If there exists an authorized prefix covering rp marked in the bm, the ROV result is *invalid*. If no covering prefix is marked in the bm or no bm is found for the given id, h^2 ROV proceeds to query the sub-trees associated with the super-prefixes of rp. This process continues until either an authorized prefix covering rp is found (resulting in *invalid*), or the root of the IP prefix trie is reached without a match (resulting in *notFound*).

As shown in Figure 5, the router inserts the three ROV rules into the STT without their authorized ASes. These entries are (58720260, 00000010), (58720260, 11001000), and (1879048336, 00110100). Since the first two entries share the same id, their bms are merged with a bitwise OR, resulting in an updated entry with the key 58720260 and the value 11001010 (i.e., 11001000 OR 00000010). For route (192.0.2.64/26, AS X), h^2 ROV calculates the id of the sub-tree as 58720260, and determines its index within the sub-tree as 3. After an SOT query fails, h^2 ROV proceeds to query the STT using the id as the key. The merged bitmap 11001010 is retrieved, and since the 3rd bit is set to 1, the ROV result for this route is *invalid*. This outcome indicates that the rp is covered but not matched by an ROV rule; otherwise, a matching ROV rule would have been found in the SOT. In contrast, route (192.0.2.0/24, AS Y) receives a *notFound* verdict due to missing matches in both SOT and STT.

4.4 Basic Design of h^2 ROV

The h^2 ROV validation process operates through two distinct phases: **matching the route** and **covering the route prefix**.

Algorithm 1: Validate a given route with h^2ROV

Input : the Subtree-Origin Table: SOT , the SubTree Table: STT , the Route: (rp, ro)

Output : the ROV result of the given route: res

```
1  $res \leftarrow NotFound$ ;  
2  $id, pos \leftarrow calculate\_ID\_AND\_POS(rp)$ ;  
3 if ( $bm_1 = SOT[id + ro]$  and  $bm_1[pos] == 1$ ) then  
4    $res \leftarrow Valid$ ;  
5   return  $res$ ;  
6 end  
7 while  $id > 0$  do  
8   if  $bm_2 = STT[id]$  and  $ExistCover(bm_2, pos)$  then  
9      $res \leftarrow Invalid$ ;  
10    return  $res$ ;  
11  end  
12   $id >>= 5$ ;  
13 end  
14 return  $res$ ;
```

First, h^2ROV determines whether a route is *valid* by querying the SOT for matching ROV rules (in Section 4.2). If the SOT query fails, h^2ROV recursively searches the STT for ROV rules that cover the route prefix to determine whether the route is *invalid* or *notFound* (in Section 4.3).

The pseudo code of h^2ROV is shown in Algorithm 1. In the worst case, h^2ROV requires one SOT query and $\lceil \frac{|rp|}{5} \rceil$ STT queries to validate a given route (rp, ro) . Therefore, the time complexity of h^2ROV for validation is $O(|rp|)$.

4.5 Optimization for IPv4 Route Validation

Obviously, the performance bottleneck of h^2ROV arises from repeated lookups in the STT, termed the backtracking issue. This issue occurs as a bitmap only encode authorized prefixes within the sub-tree, leaving coverage by other sub-trees uncertain. To tackle this challenge, we propose optimizations for IPv4 and IPv6 scenarios to reduce backtracking.

In the IPv4 scenario, to expedite the process of checking whether rp is covered, we introduce additional bitmap arrays to mark covered sub-tree root prefixes. For root prefixes at hanging levels 5, 10, 15, and 20, we construct bitmap arrays B_i of 2^i bits, respectively. For instance, B_5 has $2^5=32$ bits and is initially set to 0. These bitmap arrays are referred to as the **Level Bitmap (LB)**. For each root prefix p at these levels, we use the decimal value of first $|p|$ bits of p as its index in B_i . For example, the index of 8.0.0.0/5 in B_5 is 1(0b00001). When a prefix is authorized, we mark all covered root prefixes at these hanging levels with 1. For a given route (rp, ro) , if h^2ROV fails to match the route in the SOT, it uses the LB to check if rp is covered by other authorized prefixes, thus avoiding multiple STT queries. This optimization enables

efficient processing of routes with prefix lengths of 24 or shorter. For routes whose prefix lengths exceed 24, h^2ROV requires up to three STT queries and one LB access in the worst case. This is because sub-trees rooted at hanging levels 20, 25, and 30 may contain authorized prefixes covering rp , while the LB only records the coverage status up to hanging level 20.

To balance validation speed and memory usage, we refrain from constructing bitmap arrays for hanging levels 25 and 30 for two main factors. First, the vast majority of IPv4 prefixes have lengths of 24 or shorter [14]. Second, constructing bitmap arrays for hanging levels 25 and 30 would demand 32MB and 1GB of memory respectively. By limiting LB construction to hanging levels 5, 10, 15, and 20, h^2ROV incurs only an additional 1MB of memory overhead while reducing time complexity of validation to $O(1)$.

4.6 Optimization for IPv6 Route Validation

In the IPv6 scenario, for a route with a prefix length of 128, h^2ROV requires 26 ($\lceil \frac{128}{5} \rceil$) STT queries in the worst case. Despite applying LB, the number of STT queries merely decreases to 22 ($\lceil \frac{128-20}{5} \rceil$), which is a limited improvement in validation performance. In fact, most IPv6 routes have prefix lengths between 32 and 48 [15]. However, constructing bitmap arrays for hanging levels 30 and 45 is impractical due to the excessive storage requirements. Therefore, a more efficient optimization tailored to IPv6 scenario is necessary.

We introduce **Cover Flag (CF)**, which is the 0th bit of the bm , to indicate whether at least one super-prefix covers the sub-tree. The CF enables early termination of STT backtracking by instantly checking coverage upon retrieving bm . However, even with the CF effectively reducing the number of STT backtracking queries, 26 queries can still be required in the worst case when intermediate entries are missing from the STT. To further optimize the validation performance, we incorporate a binary search strategy. However, the binary search can only be performed if all intermediate sub-trees along the path from the root of the sub-tree containing rp to the root of the IPv6 address prefix trie exist in the STT. Therefore, before performing binary search, we ensure that all intermediate sub-trees exist in the STT by inserting placeholder entries to fill in the missing ancestor sub-trees. These placeholder entries are initialized with bitmaps set to 0, ensuring structural continuity for binary search.

For binary search on a route (rp, ro) , two pointers are initialized: the left pointer (lft) to 0 and the right pointer (rt) to $|rp|$. The middle pointer (mid) is calculated as $(lft+rt)/2$. If the bitmap corresponding to $id = ID(rp, mid)$ is absent in the STT, mid is recalculated as $(lft+mid)/2$; otherwise, if the corresponding bit in the obtained bitmap for this route is set to 1 or the CF of the obtained bitmap is set to 1, the route is determined to be *invalid* and the search terminates. If neither condition is satisfied, mid is updated to $(mid+rt)/2$

Table 1: The datasets used for evaluation.

Name	Date/Time Period	Source	Type
ROA_2024	2024/8/25	RIPE NCC	real-trace ROAs
ROA_13Y	2012/1/1~2024/1/1	RIPE NCC	real-trace ROAs
BGP_RRC00	2024/8/25	RIS RRC00	real-trace BGP updates
BGP_13Y	2012/1/1~2024/1/1	RIS RRC00	real-trace BGP updates
BGP_20C	2024/8/25	RIS 20 collectors	real-trace BGP updates
BGP_ROV	N/A	N/A	randomly-generated BGP updates
BGP_IP46	2024/8/25	RIS 20 collectors	randomly-selected BGP updates
BGP_SIZE	2024/8/25	RIS 20 collectors	randomly-selected BGP updates

Table 2: The experimental environments.

Experimental & System Evaluation	CPU	AMD EPYC 7742(2.2GHz, 128cores)
	RAM	DRAM 128G
	NIC	BCM57412 NetXtreme-E 10Gb
Emulation Platform	CPU	ARM Kunpeng-920 (344MHz, 128cores)
	RAM	RAM 560GB
	NIC	Intel Corporation 82599ES 10-Gigabit

6.1 Methodology

Evaluated schemes. We evaluate our proposed ROV scheme, h^2 ROV, and all existing ROV schemes mentioned in Section 7 except for x BGP. These schemes are represented by their core data structures namely LFFST (RTRLlib), HT (BIRD), HT+PT (BIRD), and Patricia (BGP-SRx).

Data sets. We evaluate h^2 ROV using two types of datasets: ROA and BGP update. ROA datasets, sourced from RIPE NCC [18], include the dataset ROA_13Y, comprising 13 annual sets collected on January 1st from 2012 to 2024 for scalability experiments, and the dataset ROA_2024, collected on August 25, 2024, for other experiments. For BGP update datasets, we use three categories: real-trace, randomly-generated, and randomly-selected updates. Real-trace BGP updates are obtained from RIPE Routing Information Service (RIS) [19] collectors. The dataset BGP_20C comprises BGP updates from 20 RIPE RIS collectors on August 25, 2024. The dataset BGP_13Y contains annual RRC00 BGP updates on January 1st from 2012 to 2024 for scalability experiments. The dataset BGP_RRC00 comprises BGP updates from RRC00 on August 25, 2024. Furthermore, based on the dataset ROA_2024, we generate the dataset BGP_ROV by varying the proportion of randomly generated BGP updates corresponding to each ROV result (*valid*, *invalid*, *notFound*) from 0% to 100% in 10% increments, ensuring the sum of proportions equals 1. This results in 66 sets of BGP updates. The dataset BGP_IP46 consists of 11 sets with varying IPv4/IPv6 ratios, proportionally sampled from BGP_20C. The dataset BGP_SIZE includes 3 sets of BGP updates with sizes of 10K, 100K, and 1M, randomly sampled from BGP_20C. For further details, refer to Table 1 and Appendix E.

Performance metrics. For experimental evaluation, we introduce three key performance metrics to compare ROV schemes. First, validation speed is measured as the average

number of routes validated per second, in million validations per second (MVPS). Second, memory consumption is measured in megabytes (MB). Lastly, update performance is measured as the average time taken to update an ROA. For the system implementation, we also introduce three performance metrics to evaluate the impact of ROV schemes. First, the average proportion of delay introduced by ROV in BGP decision process under normal router operation is evaluated. Then, the impact of ROV on BGP update processing speed during an update burst is measured by the average packet processing delay per second. Finally, the impact of ROV deployment on BGP convergence time is measured.

Experimental environment. For system evaluation, a small-scale RPKI and BGP testbed is constructed, with Routinator as the RP and FRRouting and BIRD as ROV-enforcing routers (abbreviated as *validators*). Furthermore, a BGP sender and receiver, both modified from FRRouting, are deployed. The sender replays BGP updates by extracting AS Path and NLRI attributes from archived BGP update files, encapsulating them into BGP updates, and sending them to the *validator* according to their timestamps. The *validator* synchronizes ROAs from the RP, performs ROV on the received BGP updates, and forwards the validated updates to the receiver, which logs the timestamp of each validated update upon arrival. Moreover, we measure BGP convergence time based on our team’s self-developed emulation platform [20]. The configuration information of the experimental environment is shown in Table 2.

6.2 Algorithm Performance Evaluation

6.2.1 Configurations of h^2 ROV

The threshold for δ determines the set of wide ROAs, influencing both validation speed and memory usage in h^2 ROV. We set the optimal threshold for both original and refined versions of h^2 ROV via experiments using datasets ROA_2024 and BGP_RRC00, considering both speed and memory costs.

As shown in Figures 7 and 8, the optimal threshold for δ in IPv4 and IPv6 scenarios for the original version of h^2 ROV is 8 and 5, respectively, and 8 and 7 for the refined version. These thresholds are used in our evaluations.

6.2.2 Performance optimization

Figures 9 and 10 show validation speed and memory cost when performing ROV on BGP updates with the original and refined versions of h^2 ROV, using datasets ROA_2024 and BGP_RRC00. The refined version achieves an increase in validation speed of 20.1% and 48.6%, respectively, for IPv4 and IPv6 compared to the original version. Correspondingly, memory usage increases by 11.4% for IPv4 and 72% for IPv6 but remains within acceptable limits. Furthermore, we evaluate the validation speed of both versions of h^2 ROV using BGP_20C. The experimental results in Appendix D.3 demonstrate that

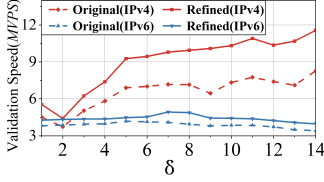


Figure 7: The validation speed with different thresholds for δ .

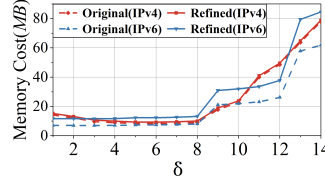


Figure 8: The memory cost with different thresholds for δ .

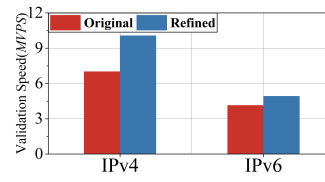


Figure 9: The validation speed of two versions of h^2ROV .

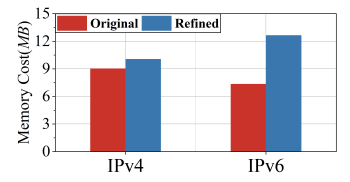


Figure 10: The memory cost of two versions of h^2ROV .

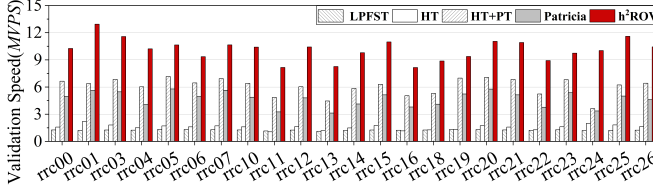


Figure 11: The validation speed of five ROV schemes in IPv4.

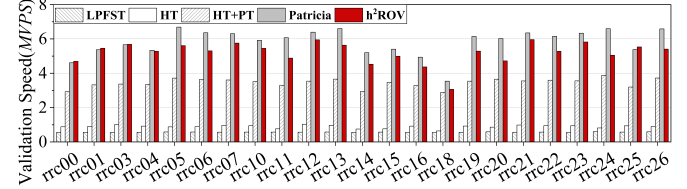


Figure 12: The validation speed of five ROV schemes in IPv6.

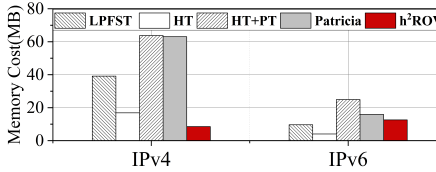


Figure 13: The memory cost.

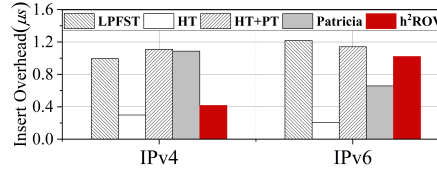


Figure 14: The average insertion time.

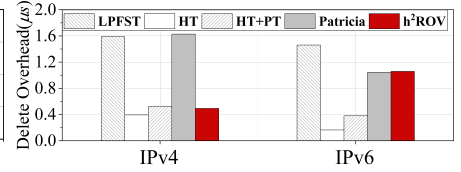


Figure 15: The average deletion time.

the refined version exhibits a clear advantage in validation speed. Therefore, we adopt the refined version for further experiments and system implementation.

6.2.3 Comparison with existing ROV schemes

Validation speed. We evaluate the validation speed of five schemes using datasets ROA_2024 and BGP_20C, as shown in Figures 11 and 12. In the IPv4 scenario, h^2ROV achieves validation speed of 8.1 to 12.9 MVPS across different BGP collectors. Its validation speed is $6.8\times$ to $9.8\times$, $4.3\times$ to $6.2\times$, $1.7\times$ to $2.4\times$, and $2.2\times$ to $2.9\times$ that of LPFST, HT, HT+PT, and Patricia, respectively. These results demonstrate a significant performance advantage of h^2ROV in the IPv4 scenario. In the IPv6 scenario, h^2ROV achieves validation speed of 3 to 5.96 MVPS. While it outperforms three ROV schemes—LPFST, HT, and HT+PT—it is slightly slower than Patricia across most collectors.

Memory consumption. Figure 13 shows the memory consumption of five ROV schemes. In the IPv4 scenario, h^2ROV consumes 8.5 MB memory, reducing memory consumption by 78.1%, 49.3%, 86.6%, and 86.4% compared to LPFST, HT, HT+PT, and Patricia, respectively. In the IPv6 scenario, h^2ROV consumes 12.6 MB memory. Although it uses less memory than HT+PT and Patricia, it requires more memory than LPFST and HT. Overall, h^2ROV achieves combined memory consumption of 21.1 MB for IPv4 and IPv6, exceeding HT by only 0.2 MB while remaining significantly lower than other evaluated schemes.

Update performance. We evaluate the update performance of five ROV schemes. For insertion evaluation, we use the

dataset ROA_2024. For deletion evaluation, we randomly select part of the dataset ROA_2024 for removal. Figures 14 and 15 show that the update performance of h^2ROV is better than other ROV schemes except for HT in the IPv4 scenario. However, in the IPv6 scenario, h^2ROV outperforms only LPFST. Overall, h^2ROV completes an ROV rule insertion or deletion in under $1\mu s$. Appendix C indicates that the frequency of ROA updates per second is significantly low. Consequently, the ROV update performance does not pose a bottleneck for route validation.

6.3 Scalability of h^2ROV

6.3.1 Volume of ROAs

With the ROAs and BGP updates collected from the operational networks every year (datasets ROA_13Y and BGP_13Y), we measure the validation speed and memory consumption of the five schemes.

As shown in Figure 16, h^2ROV demonstrates a more stable and upward trend in validation speed for IPv4 BGP updates, whereas the validation speeds of other ROV schemes decline over time, except for HT. From the trend demonstrated in Figure 17, we can deduce that the superiority of h^2ROV over other ROV schemes will become clearer with the increase in the number of ROAs in IPv6 scenario. In terms of memory cost, h^2ROV clearly outperforms other schemes in IPv4, as shown in Figure 18, and for IPv6 (Figure 19), its memory usage increases but at a slower rate than Patricia and HT+PT.

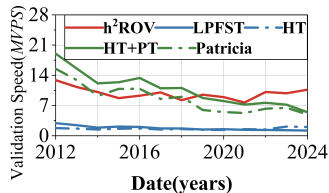


Figure 16: The validation speed at varying times in IPv4.

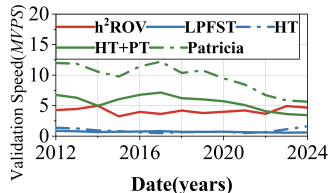


Figure 17: The validation speed at varying times in IPv6.

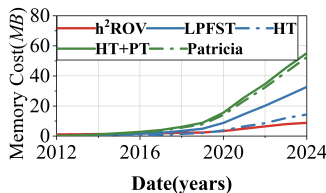


Figure 18: The memory cost at varying times in IPv4.

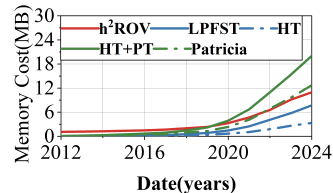


Figure 19: The memory cost at varying times in IPv6.

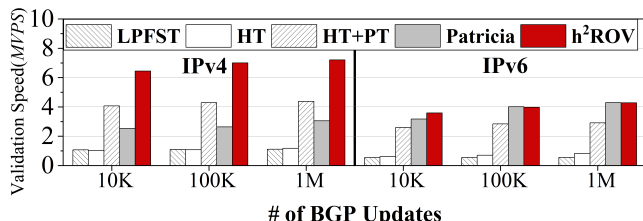


Figure 20: The validation speed with different volume of BGP updates.

6.3.2 Volume of BGP updates

To investigate the impact of BGP update volume on ROV validation speed, we use the datasets ROA_2024 and BGP_SIZE. As depicted in Figure 20, our results demonstrate a consistent pattern across all ROV schemes: validation speed gradually increases with the growing number of BGP updates. This is because extracting more BGP updates increases the cache hit rate due to redundancy in BGP updates, leading to a gradual rise in validation speed.

6.3.3 Distribution of validity states

As RPKI adoption grows, the increasing issuance of ROAs will alter the distribution of route validity states. To measure the impact of these changes on validation speed, we evaluate ROV schemes using the datasets ROA_2024 and BGP_ROV.

The experimental results are shown in Figure 21 and Figure 22. LPFST, HT+PT, and Patricia show a decreasing trend in validation speed compared to the current state (nearly 50% valid), as the ratio of valid results increases in both IPv4 and IPv6. These ROV schemes achieve the fastest validation when the ratio of *notFound* results is highest. In contrast, HT shows an increasing trend in validation speed as the ratio of valid results rises. However, it remains slower than h²ROV due to the *AB Model*, which prevents HT from finding a matching ROA directly. Overall, h²ROV outperforms all other ROV schemes when the ratio of *notFound* results is below 40%, a scenario reflecting the future RPKI deployment levels.

6.3.4 Incremental IPv6 deployment

To analyze the impact of the incremental IPv6 deployment, we use datasets ROA_2024 and BGP_IP46 to measure the validation performance of five ROV schemes under different proportions of IPv4 and IPv6 BGP updates.

As shown in Figure 23, as the proportion of IPv6 BGP updates increases, a noticeable decline in validation speed is observed for h²ROV, LPFST, HT, and HT+PT, while Patricia shows an increase in validation speed. The validation speed of h²ROV decreases from 6.53 MVPS to 3.99 MVPS as the IPv6 proportion rises from 0% to 100%. However, it still demonstrates significantly better validation performance than LPFST, HT, and HT+PT, except at 100% IPv6 updates, where its speed is just 0.028 MVPS lower than Patricia.

6.4 System Performance Evaluation

In the system evaluation, we first measure the average delay introduced by ROV in the *BGP decision process* during normal BGP update reception. We implement five ROV schemes in FRRouting and BIRD and evaluate them using datasets ROA_2024 and BGP_RRC00. Based on experimental results, h²ROV introduces the least average delay to the *BGP decision process* at 19.8%(5.9%), achieving reduction of 10.4%(13.3%) ~ 56.1%(61.4%) compared to other ROV schemes in FRRouting (BIRD).

Next, we replicate a real update burst event to evaluate the impact of ROV on average BGP update processing delay during sudden high-volume updates. Using 340K BGP updates from BGP_RRC00 replayed in a small-scale testbed, Figures 24 and 25 reveal a significant increase in average processing delay during the burst, exacerbated by ROV enforcement. Overall, h²ROV exhibits the least impact compared to other ROV schemes. When the average processing delay reached its peak (282nd second), h²ROV introduces the smallest average processing delay, reducing it by 48.8%(46.9%) ~ 83.2%(70.9%) compared to other schemes with FRRouting (BIRD).

Finally, we evaluate the impact of ROV on BGP convergence time in the fully ROV-deployed scenario across real-world and generated topologies, based on our team's self-developed emulation platform [20]. The real-world topologies are constructed using data from the CAIDA's AS relationships [21]. The first real topology includes the top 100 ranked ASes, where ROV deployment is assumed to be more prevalent, following suggestions from studies [3, 22] to prioritize high-ranking or Tier-1 ASes. The second real topology is the South Korean network, chosen for its regional completeness, supported scale in the emulation platform, and the real-world KT network outage incident [23]. We source the top 100 ranked ASes from AS Rank [24] and Korea ASes

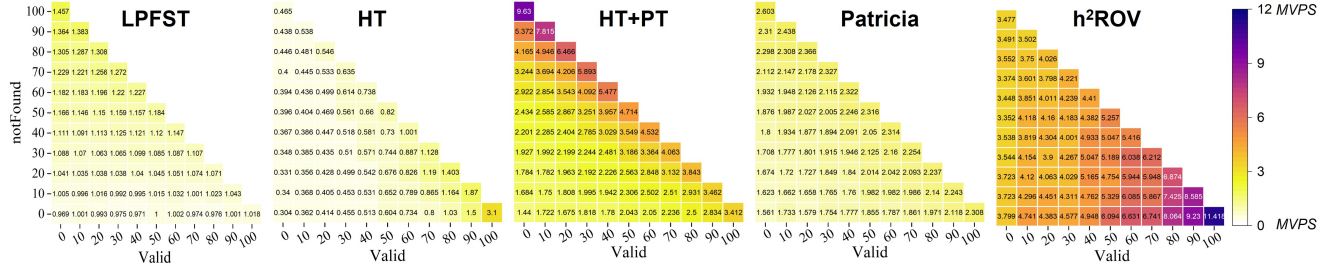


Figure 21: The validation speed of five schemes with different distributions of validity states of routes in IPv4.

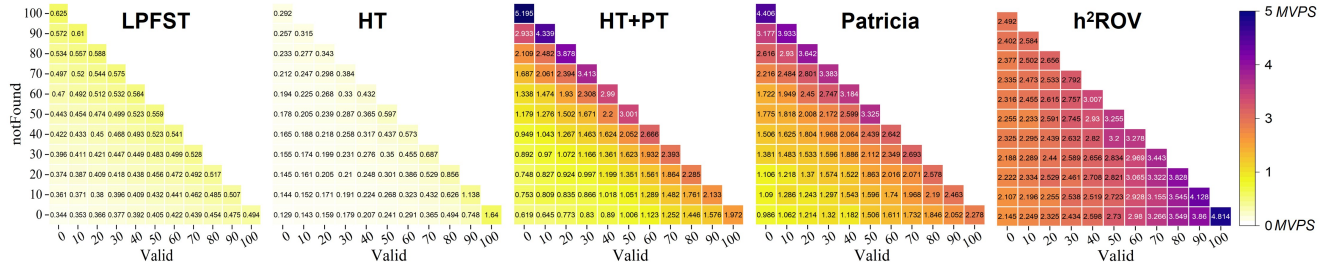


Figure 22: The validation speed of five schemes with different distributions of validity states of routes in IPv6.

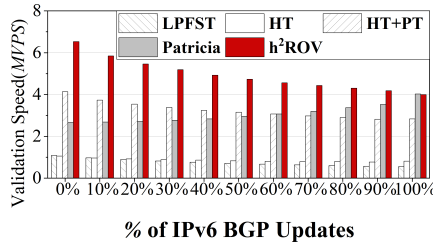


Figure 23: The validation speed with different IPv4/IPv6 updates ratio.

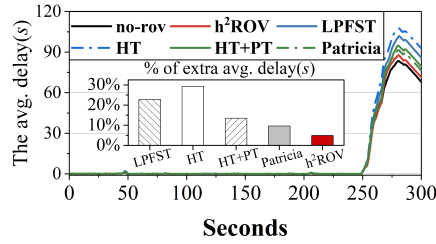


Figure 24: The average processing delay with FRRouting.

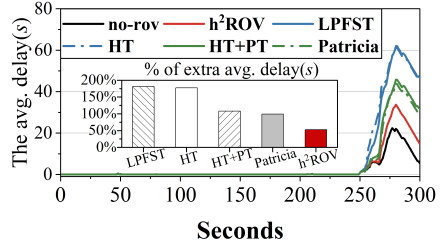


Figure 25: The average processing delay with BIRD.

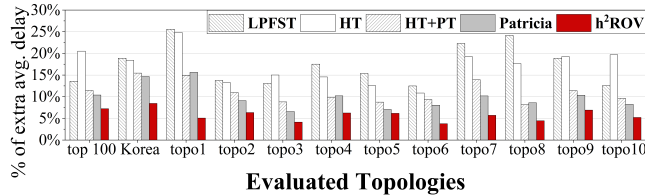


Figure 26: The average proportion of ROV delay in the BGP convergence time.

from Hurricane Electric [25]. Then, we derive sub-graphs containing top 100 ranked ASes and Korea ASes from the complete topology built using CAIDA AS Relationship data on June, 2024 as top 100 topology and Korea topology. The top 100 topology has 100 nodes and 1,916 edges, and the Korea topology has 803 nodes and 1,413 edges. Furthermore, ten AS topologies with statistical parameters aligned with known AS topologies are constructed using the Hyperbolic Graph Generator [26]. Each topology has 1K ASes, an average node degree of 6.4 which is the value observed in the CAIDA AS level topology [21] on June 2024, and a power-law degree distribution with an exponent of 2.1, as in [27, 28]. The AS transit delay is set to 1 ms for all topologies. Once the BGP nodes in the topology are initiated and neighbor relationships are established, BGP updates from the dataset BGP_RRC00 are

injected into the node with the highest connectivity. The BGP convergence time is measured from the start of route reception until all routing tables stabilize without further changes [29]. We evaluate the impact of ROV implementation on convergence time under both normal route updates and update burst scenarios. In the normal scenario, BGP updates were transmitted according to the timestamps in the dataset, whereas in the burst scenario, all updates were transmitted simultaneously. Under normal operating conditions, the convergence time increased only by 1.1% ~ 9.5%. However, during the update burst scenario, ROV-induced routing convergence delays were substantial, reaching up to 8.0% ~ 25.6%. As shown in Figure 26, h²ROV increases the convergence time by only 3.8% ~ 8.5%. Specifically, in real-world network topologies, it mitigates ROV-induced routing convergence delays by up to 30.4% ~ 64.7%, while in generated topologies, the mitigation reaches up to 12.5% ~ 81.6%. In summary, h²ROV exhibits the minimal impact on convergence time.

7 Related Work

Measurement of ROV enforcement. The quantification of ROV adoption is important to assess its effect in protecting the

inter-domain routing system. Generally, measurement of ROV enforcement can be conducted with uncontrolled BGP data analysis [22] or enhanced passive monitoring [30]. On the other hand, active probing techniques [3, 31] have been used to improve the accuracy. While hybrid approaches [32] and multi-plane analysis [33] achieve better trade-offs between accuracy and efficiency.

Boosting the security benefit of ROV. Partial RPKI deployment limits the security benefit of ROV. It has been demonstrated that Tier-1 ISP adoption [22, 34] and transit provider prioritization [3, 30] maximize effect of ROV in stopping route hijackings. While ROV++ [35] introduces a series of optimizations that can prevent over 90% of subprefix hijackings with a low rate of ROV adoption. Besides, provider-origin validation [4] has been proposed towards asymmetric trust relationships, and RPKIN [36] enables collaborative hijack containment via neighbor notifications.

Advances in ROV implementation. IETF has standardized RPKI ROV [9] for years, its reference implementation using linear search has been outdated. RTRLib [6] employs a Longest Prefix First Search Tree (LPFST) to simplify the validation process at the cost of poor scalability. BIRD [7] introduces a hybrid approach combining prefix trie and hash tables together to improve the comprehensive performance. While BGP-SRx [8] employs a Patricia tree to accelerate the validation, outperforming other solutions.

8 Discussion

8.1 Performance Limitation of h^2 ROV in IPv6

The basic h^2 ROV scheme incurs linearly scaling hash probes on the STT relative to route prefix length. In IPv4, our level-bitmap optimization bounds this to a constant three probes, yielding $O(1)$ time complexity. In IPv6, however, the 128-bit address space precludes direct application of level bitmaps. While our binary search approach reduces the complexity to $O(\log n)$, it achieves less pronounced speedups than IPv4's constant-time validation. Consequently, h^2 ROV's performance in IPv6 shows marginal improvements over existing schemes; it is even slightly slower than Patricia in some cases. Our future work will explore heuristic strategies [37] to bridge this performance gap, potentially aligning IPv6 efficiency with IPv4's deterministic guarantees.

8.2 Hardware Implementation Guidelines

Modern hardware routers implement control plane functions—including BGP decision processes and RPKI-based validation—via software stacks [38, 39] running on general-purpose CPUs. Critically, these workflows exhibit structural parity with software router implementations. Consequently, empirical insights we have learned from integrating h^2 ROV into software routers directly inform its deployment in

hardware environments. Two implementation requirements persist across both platforms: (1) augmenting the PDU handler to construct ROV rules with the bitmap encoding, and (2) substituting the legacy route origin validator with h^2 ROV's validator module that can boost the performance.

8.3 Incremental Deployment of the AP Model

The *AP model* advances the conventional *AB* framework through a redefined validation process with structured encoding format of validation rules. Its deployment occurs in two backward-compatible phases. In the first phase with legacy compatibility, routers operate without RTR protocol modifications, but require h^2 ROV's validator as well as its encoder. While the encoder introduces marginal overhead, the cost is dwarfed by h^2 ROV's aggregate performance gains. Crucially, adoption of the *AP model* remains optional per router, allowing autonomous cost-benefit analysis by operators. In the second phase, when RTR PDUs with bitmap-encoded ROV rules become available⁵, h^2 ROV's encoder can be bypassed. Critically, both phases can operate concurrently within a single administrative domain, enabling incremental *AP model* adoption without stranding legacy infrastructure.

9 Conclusion

While ROV remains a critical defense against BGP hijacking, existing AB-model systems struggle to scale with the increasing volume of ROV rules and velocity of BGP updates. Our proposed AP model and its instantiation, h^2 ROV, address this challenge through prefix-granular validation, reducing IPv4 lookup complexity to $O(1)$ via level-bitmap optimization. However, IPv6's sparse address hierarchy limits these gains, as h^2 ROV's binary search adaptation achieves only $O(\log n)$ complexity. The integration of h^2 ROV into BIRD and FRRouting demonstrates its compatibility with classic router control planes, providing a practical pathway for hardware router adoption. The source code of the reference implementation of h^2 ROV is available at <https://github.com/FIRLab-CNIC/h-2ROV>.

Acknowledgment

We sincerely thank our shepherd Hongqiang Liu and the anonymous reviewers for their insightful and constructive suggestions. This work was supported by the National Key R&D Program of China under Grant No.2022YFB3104800 and the National Natural Science Foundation of China under the Grant No. 62072430. The corresponding authors of this paper are Yanbiao Li and Gaogang Xie.

⁵How this kind of PDUs are produced depends on the deployment of hanging ROAs [12] and is out of the scope of this work.

References

- [1] Yakov Rekhter, Tony Li, and Susan Hares. Rfc 4271: A border gateway protocol 4 (bgp-4), 2006.
- [2] Matt Lepinski and S Kent. Rfc 6480: an infrastructure to support secure internet routing, 2012.
- [3] Weitong Li, Zhexiao Lin, Md Ishtiaq Ashiq, Emile Aben, Romain Fontugne, Amreesh Phokeer, and Taejoong Chung. Rovista: Measuring and analyzing the route origin validation (rov) in rpki. In *Proceedings of the 2023 ACM on Internet Measurement Conference*, pages 73–88, 2023.
- [4] Lancheng Qin, Li Chen, Dan Li, Honglin Ye, and Yutian Wang. Understanding route origin validation (rov) deployment in the real world and why manrs action 1 is not followed. NDSS, 2024.
- [5] Bgpmon. <https://www.bgpmon.net>, 2024.
- [6] Matthias Wählisch, Fabian Holler, Thomas C. Schmidt, and Jochen H. Schiller. RTRlib: An Open-Source library in c for RPKI-based prefix origin validation. In *6th Workshop on Cyber Security Experimentation and Test (CSET 13)*, Washington, D.C., August 2013. USENIX Association.
- [7] The bird internet routing daemon. <https://bird.network.cz/>, 2024.
- [8] Oliver Borchert, Kyehwan Lee, Kotikalapudi Sriram, Douglas Montgomery, Patrick Gleichmann, and Mehmet Adalier. Bgp secure routing extension (bgp-srx): Reference implementation and test tools for emerging bgp security standards. 2021.
- [9] P Mohapatra, J Scudder, D Ward, R Bush, and R Austein. Rfc 6811: Bgp prefix origin validation, 2013.
- [10] J Snijders, B Maddison, M Lepinski, D Kong, and S Kent. Rfc 9582: A profile for route origin authorizations (roas), 2024.
- [11] R. Bush and R. Austein. The Resource Public Key Infrastructure (RPKI) to Router Protocol, Version 1. RFC 8210, September 2017.
- [12] Yanbiao Li, Hui Zou, Yuxuan Chen, Yinbo Xu, Zhuoran Ma, Di Ma, Ying Hu, and Gaogang Xie. The hanging roa: A secure and scalable encoding scheme for route origin authorization. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*, pages 21–30. IEEE, 2022.
- [13] Michael Fredman and Michael Saks. The cell probe complexity of dynamic data structures. In *Proceedings of the twenty-first annual ACM symposium on Theory of computing*, pages 345–354, 1989.
- [14] Bgp in 2023 – have we reached peak ipv4? <https://labs.apnic.net/index.php/2024/01/05/bgp-in-2023-have-we-reached-peak-ipv4/>, 2024.
- [15] Donghong Jiang, Yanbiao Li, Yuxuan Chen, Jing Hu, Yi Huang, and Gaogang Xie. Heuristic binary search: Adaptive and fast ipv6 route lookup with incremental updates. In *Proceedings of the 7th Asia-Pacific Workshop on Networking*, pages 47–53, 2023.
- [16] Frrouting project. <https://frrouting.org/>, 2024.
- [17] Bgp in 2023 – bgp updates. <https://labs.apnic.net/index.php/2024/01/08/bgp-in-2023-bgp-updates/>, 2024.
- [18] Index of /rpki/. <https://ftp.ripe.net/rpki/>, 2024.
- [19] Routing information service (ris). <https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris/>, 2024.
- [20] Sernes. <https://www.sernes.cn/>, 2024.
- [21] The caida as relationships dataset. <https://www.caida.org/catalog/datasets/as-relationships>, 2024.
- [22] Yossi Gilad, Avichai Cohen, Amir Herzberg, Michael Schapira, and Haya Shulman. Are we there yet? on rpki’s deployment and security. *Cryptology ePrint Archive*, 2016.
- [23] South korean telco goes down, blames ddos attack, later admits its bgp broke. https://www.theregister.com/2021/10/26/kt_outage_bgp_not_ddos/, 2021.
- [24] As rank. <https://asrank.caida.org/>, 2024.
- [25] Hurricane electric. <https://bgp.he.net/>, 2024.
- [26] Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguná. Hyperbolic geometry of complex networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 82(3):036106, 2010.
- [27] Thomas Holterbach, Stefano Vissicchio, Alberto Dainotti, and Laurent Vanbever. Swift: Predictive fast reroute. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 460–473, 2017.
- [28] Thomas Alfroy, Thomas Holterbach, Thomas Krenc, KC Claffy, and Cristel Pelsser. The next generation of bgp data collection platforms. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 794–812, 2024.

- [29] Almekdad Abaid, Mahmoud Hraib, Abdulrazig Ben Ghazzi, and Salem Sati. Convergence time analysis of border gateway protocol using gns3. In *2021 IEEE 1st International Maghreb Meeting of the Conference on Sciences and Techniques of Automatic Control and Computer Engineering MI-STA*, pages 689–694. IEEE, 2021.
- [30] Cecilia Testart, Philipp Richter, Alistair King, Alberto Dainotti, and David Clark. To filter or not to filter: Measuring the benefits of registering in the rpki today. In *Passive and Active Measurement: 21st International Conference, PAM 2020, Eugene, Oregon, USA, March 30–31, 2020, Proceedings 21*, pages 71–87. Springer, 2020.
- [31] Andreas Reuter, Randy Bush, Italo Cunha, Ethan Katz-Bassett, Thomas C Schmidt, and Matthias Wählisch. Towards a rigorous methodology for measuring adoption of rpki route validation and filtering. *ACM SIGCOMM Computer Communication Review*, 48(1):19–27, 2018.
- [32] Nils Rodday, Ítalo Cunha, Randy Bush, Ethan Katz-Bassett, Gabi D Rodosek, Thomas C Schmidt, and Matthias Wählisch. Revisiting rpki route origin validation on the data plane. In *Proc. of Network Traffic Measurement and Analysis Conference (TMA), IFIP*, 2021.
- [33] Tomas Hlavacek, Amir Herzberg, Haya Shulman, and Michael Waidner. Practical experience: Methodologies for measuring route origin validation. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 634–641. IEEE, 2018.
- [34] Tomas Hlavacek, Haya Shulman, Niklas Vogel, and Michael Waidner. Keep your friends close, but your routeservers closer: Insights into {RPKI} validation in the internet. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 4841–4858, 2023.
- [35] Reynaldo Morillo, Justin Furuness, Cameron Morris, James Breslin, Amir Herzberg, and Bing Wang. Rov++: Improved deployable defense against bgp hijacking. In *NDSS*, 2021.
- [36] Man Zeng, Xiaohong Huang, Pei Zhang, Dandan Li, and Kun Xie. Improving prefix hijacking defense of rpki from an evolutionary game perspective. *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [37] Donghong Jiang, Yanbiao Li, Yuxuan Chen, Jing Hu, Yi Huang, and Gaogang Xie. Heuristic binary search: Adaptive and fast ipv6 route lookup with incremental prefix updates. *IEEE/ACM Transactions on Networking*, 2024.

- [38] Examine border gateway protocol frequently asked questions. <https://www.cisco.com/c/en/us/support/docs/ip/border-gateway-protocol-bgp/5816-bgpfaq-5816.html>, 2024.
- [39] H3c routers document shelf-6w100. https://www.h3c.com/en/Support/Resource_Center/EN/Home/Routers/00-Public/Doc_Sets/Doc_Shelf/H3C_Routers_Document_Shelf-13071/?CHID=900688, 2024.
- [40] index of rpki-views. <https://www.rpki-views.org/>, 2024.

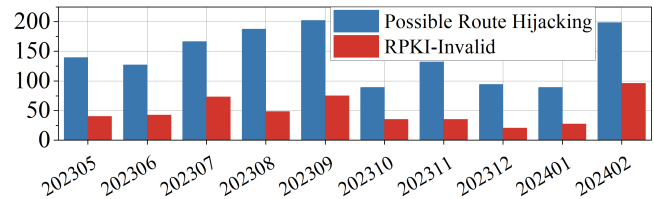


Figure 27: The number of hijacked route prefixes and those protected by ROAs.

A ROA-Protected yet Hijacked Route Prefixes

This appendix provides supplementary results for the measurement described in Section 1.

We measured the number of hijacked prefixes and the number of prefixes for which ROAs were issued. BGPMon [5], a well-known BGP anomaly detection platform⁶, reports daily hijacking events. We use reported hijacking events collected from May 1, 2023–February 29, 2024, to measure the number of hijacked prefixes. Then, we identify whether these hijacked route prefixes are protected by RPKI using a set of ROAs archived by RIPE NCC on the same day. As shown in Figure 27, 21% ~ 48% of route prefixes are protected by ROAs but are still hijacked, primarily due to the low ROV adoption rate.

B Theorem Proof

This appendix provides proof of Theorem 1 introduced in Section 3.3.

Definition 8 Given a prefix p , the first n bits of p are denoted by $BIT(p, n)$.

Definition 9 $p_i \subseteq p_j$ if and only if $BIT(p_i, |p_j|) = BIT(p_j, |p_j|)$ and $|p_j| \leq |p_i|$.

Proof. If $rp \not\subseteq R.pfx$, there are two cases according to Definition 9:

⁶It has ceased to offer free service to the public since February 2024.

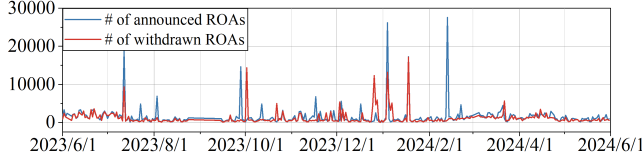


Figure 28: The number of daily announced and withdrawn ROAs.

1. $|R.pfx| > |rp|$, or
2. $BIT(R.pfx, l) \neq BIT(rp, l)$, where $l = |R.pfx|$

For the first case, according to the Definitions 6 and 9, $\forall p \in AP_{set}(R), |p| \geq |R.pfx|$, thus we can get $\forall p \in AP_{set}(R), |p| > |rp|$. Therefore, $\forall p \in AP_{set}(R), rp \not\subseteq p$.

For the second case, according to the Definitions 6 and 9, $\forall p \in AP_{set}(R), BIT(p, l) = BIT(R.pfx, l)$, where $l = |R.pfx|$, thus we can get $\forall p \in AP_{set}(R), BIT(p, l) \neq BIT(rp, l)$. Therefore, $\forall p \in AP_{set}(R), rp \not\subseteq p$.

To sum up, in both cases, rp cannot be a sub-prefix of any authorized prefix in $AP_{set}(R)$. Therefore, we conclude

$$rp \not\subseteq R.pfx \implies \forall p \in AP_{set}(R), rp \not\subseteq p. \quad (4)$$

If $\forall p \in AP_{set}(R), rp \not\subseteq p$, according to Definition 7, we have $R.pfx \in AP_{set}(R)$, then $rp \not\subseteq AB.pfx$. Therefore, we can obtain

$$\forall p \in AP_{set}(R), rp \not\subseteq p \implies rp \not\subseteq R.pfx \quad (5)$$

By combining Eqs. 4 and 5, we prove Theorem 1.

C The Number of Daily Updated ROAs

This appendix provides additional results regarding the number of ROA updates, supplementing the analysis presented in Section 6.2.3 of the main text.

In this section, we measured the number of daily updated ROAs, including both announced ROAs and withdrawn ROAs. We collected the updated ROAs at 30-minute intervals from June 1, 2023 to May 31, 2024 provided by RPKIViews [40]. Then we calculated the number of updated ROAs generated in each interval and sum them up to obtain the total number of updates per day. The result is shown in Figure 28. Obviously, the number of daily updated ROAs is relatively small.

D More Experimental Results

D.1 Memory Usage Compared to RIB

This appendix provides additional results regarding the memory usage of five ROV schemes compared to RIB, supplementing the analysis presented in Section 1.

Table 3: The memory consumption of ROAs of five schemes compared to that of RIB in the current scenarios.

the size of ROAs (MB)			the size of RIB (MB)
	current	future	current
LPFST	48.79	109.40	111.92 (FRRouting) 102.56 (BIRD)
HT	20.97	43.31	
HT+PT	88.70	199.25	
Patricia	78.98	179.00	
h²ROV	21.19	23.25	

To evaluate the additional memory overhead introduced by ROV across five schemes, we conducted a comparative analysis in two scenarios: the current real-world environment and a projected future scenario reflecting increased RPKI adoption. For the current scenario, we used the dataset ROA_2024 to calculate the size of ROV rules. In the future scenario, we assume that all route prefixes will be authorized by ROAs. Therefore, we used BGP updates in dataset BGP_RRC00 as a reference for the issued ROAs. In addition, we use BGP RIB from RRC00 on August 25, 2024 to evaluate the size of RIB. In both scenarios, we evaluated the memory overhead of five schemes used for ROV rule maintenance on two widely-used software routers: FRRouting and BIRD. As shown in Table 3, the memory requirements of existing ROV schemes are significantly lower than those of the RIB in the current environment. However, as RPKI adoption progresses, the memory consumption of these schemes (except for h²ROV) may become comparable to or even significantly exceed that of the RIB. It is noteworthy that h²ROV maintains consistently minimal memory overhead in both current and future scenarios, representing a significant improvement over existing solutions.

D.2 Stability of Results

This section explains why the average of 100 runs was used as the experimental result at the beginning of Section 6.

To evaluate the performance stability of all ROV schemes, both implementations were subjected to 100 independent experimental trials with datasets BGP_RRC00 and ROA_2024, and visualized the results using box plots. As depicted in Figure 29 and Figure 30, the box lengths of all box plots are relatively short, and the median is close to the mean, indicating stable validation speed across trials. Therefore, all experimental results presented in this paper are based on the average of 100 runs.

D.3 Original VS. Refined

This appendix provides additional results for the experiments described in Section 6.2.2.

We compare the validation speeds of the original and refined versions of h²ROV using the BGP_20C dataset. The results, shown in Figures 31 and 32, clearly indicate that the

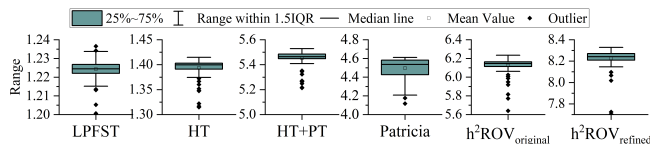


Figure 29: The validation speed (MVPS) distribution across 100 runs in IPv4.

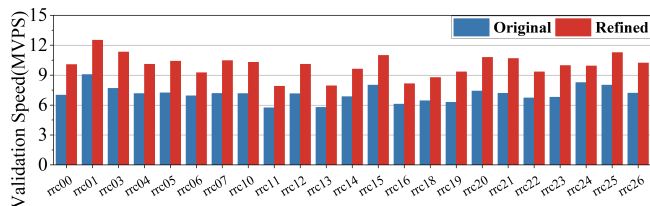


Figure 31: The validation speed of original and refined versions of h^2ROV across different sets of BGP updates in IPv4.

refined version consistently outperforms the original version across different collectors.

E More Details about Datasets

This appendix provides further details regarding datasets utilized in the experimental evaluation presented in Section 6.1.

The ROA datasets comprise ROA_13Y and ROA_2024, both sourced from RIPE NCC. The dataset ROA_13Y contains ROAs collected annually on January 1 from 2012 to 2024, while the dataset ROA_2024 contains ROAs from August 25, 2024. The BGP datasets consist of BGP_RRC00, BGP_13Y, BGP_20C, BGP_ROV, BGP_IP46, and BGP_SIZE. The dataset BGP_RRC00 contains BGP updates collected by RRC00 on August 25, 2024. The dataset BGP_13Y contains BGP updates provided by RRC00 on January 1 of each year between 2012 and 2024. The dataset BGP_20C contains BGP updates collected from 20 RIPE RIS collectors on August 25, 2024. The dataset BGP_ROV is randomly generated based on ROA_2024, forming 66 subsets with 10% incremental ROV result proportions, each containing 100K updates. These randomly generated BGP updates are constructed as follows: the origin AS in each BGP update is randomly generated, and the IP prefix is generated by first creating a random 32-bit or 128-bit IP address, then selecting a mask length at random, and finally setting the bits beyond the mask length to 0. The dataset BGP_IP46 is derived by sampling 400K updates from BGP_20C and adjusting the proportion of IPv4 and IPv6 updates in 10% increments, producing 11 subsets. Similarly, The dataset BGP_SIZE constructed from BGP_20C but varying the number of BGP updates to three predefined sizes: 10K, 100K, and 1,000K. Among these, BGP_RRC00, BGP_13Y, and BGP_20C are based on real-world BGP traces, whereas BGP_ROV, BGP_IP46, and BGP_SIZE are either randomly generated or selected according to predefined rules. Details about

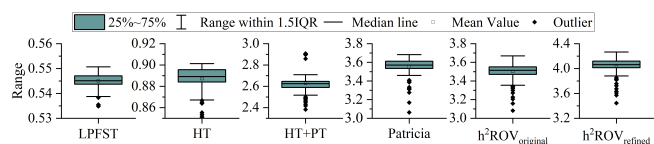


Figure 30: The validation speed (MVPS) distribution across 100 runs in IPv6.

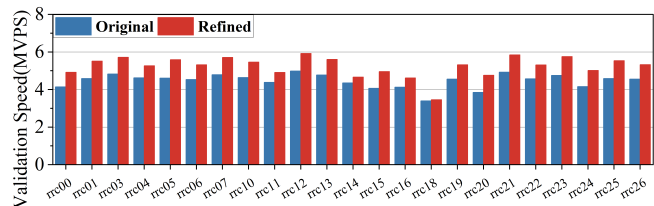


Figure 32: The validation speed of original and refined versions of h^2ROV with different sets of BGP updates in IPv6.

BGP_RRC00, BGP_13Y, and BGP_20C are presented in Table 4.

Table 4: The details about datasets.

Dataset	Date	# of IPv4 entries	# of IPv6 entries	Source
ROA_13Y	2012/1/1	1282	292	ARIN, RIPENCC, APNIC, LACNIC, AFRINIC
	2013/1/1	3729	753	
	2014/1/1	7538	1301	
	2015/1/1	13852	2331	
	2016/1/1	25714	3599	
	2017/1/1	46143	5638	
	2018/1/1	77960	14557	
	2019/1/1	59924	10127	
	2020/1/1	108419	18370	
	2021/1/1	184381	30267	
	2022/1/1	256707	50590	
	2023/1/1	340052	73849	
	2024/1/1	405157	92160	
ROA_2024	2024/8/25	487751	116911	
BGP_RRC00	2024/8/25	473412	507589	RRC00
BGP_13Y	2012/1/1	92486	547	RRC00
	2013/1/1	26813	3545	
	2014/1/1	55163	1283	
	2015/1/1	29280	100109	
	2016/1/1	56114	8484	
	2017/1/1	129369	17985	
	2018/1/1	24043	9662	
	2019/1/1	419878	27081	
	2020/1/1	1563401	560336	
	2021/1/1	732461	622754	
	2022/1/1	395531	321171	
	2023/1/1	246565	469999	
	2024/1/1	542245	868546	
BGP_20C	2024/8/25	473412	507589	RRC00
		548969	243079	RRC01
		438326	264281	RRC03
		47883	26104	RRC04
		111812	78794	RRC05
		45943	29425	RRC06
		146265	80305	RRC07
		113893	63433	RRC10
		222002	62804	RRC11
		468027	280173	RRC12
		119613	50995	RRC13
		94229	58061	RRC14
		200757	112557	RRC15
		51621	31763	RRC16
		3619	3198	RRC18
		131208	19635	RRC19
		292710	261981	RRC20
		212891	107878	RRC21
		15089	23071	RRC22
		155382	84208	RRC23
		340887	114207	RRC24
		806405	638830	RRC25
		51992	27539	RRC26